

CESI — Concepteur Développeur d'Applications

Bloc 3 — Déployer et sécuriser les applications informatiques

Dossier Technique

CESIZen

Plateforme de bien-être mental du Ministère

Présenté par : **Louis Beaujoin**

Promotion : **2024-2025**

Date : **Juillet 2026**

Dépôt GitHub : github.com/louisbeaujoin/cesizen

Sommaire

1. Introduction et contexte

2. Plan de déploiement

1. Description des trois environnements et ressources
2. Architecture technique
3. Stratégie de versioning
4. Pipeline CI/CD GitHub Actions
5. Configuration Docker — environnement de test
6. Plan de déploiement étape par étape
7. Procédure de rollback

3. Plan de maintenance

1. Outil de ticketing
2. Méthodologie et SLA
3. Veille technologique

4. Plan de sécurisation

1. Analyse OWASP
2. Matrice de risques
3. Actions correctives et préventives
4. Volet RGPD
5. Plan de gestion de crise
6. Bonnes pratiques de développement

5. Conclusion

1. Introduction et contexte du projet

1.1 Présentation

CESIZen est une plateforme web de bien-être mental commandée par un Ministère fictif. Elle propose des exercices de respiration guidés et des pages d'information sur la santé mentale, accessibles à tous les citoyens, avec un espace personnel pour les utilisateurs inscrits et un back-office pour les administrateurs.

Paramètres du projet

Budget : **75 000 € TTC** | Durée : **12 mois** | Maître d'ouvrage : **Direction Numérique du Ministère** | Prestataire : **Louis Beaujoin, développeur CDA**

1.2 Stack technique

Composant	Technologie	Version
Framework back-end	Laravel	13
Langage	PHP	8.3
CSS / Build	Tailwind CSS + Vite	4 / 7
Base de données (dev)	MySQL via WAMP	8.0
Tests automatisés	Pest / PHPUnit	4.x
Application mobile	Android natif (Kotlin)	Composant séparé

1.3 Fonctionnalités

- **Authentification** : inscription, connexion, profil, changement de mot de passe
- **Pages d'information** : articles sur la santé mentale, consultables sans connexion
- **Exercices de respiration** : widget animé (inspiration / apnée / expiration), sauvegarde de sessions
- **Back-office admin** : gestion des utilisateurs, des pages d'information et des exercices
- **Application Android** : composant natif Kotlin, consomme l'API REST de CESIZen — hors périmètre Docker/CI

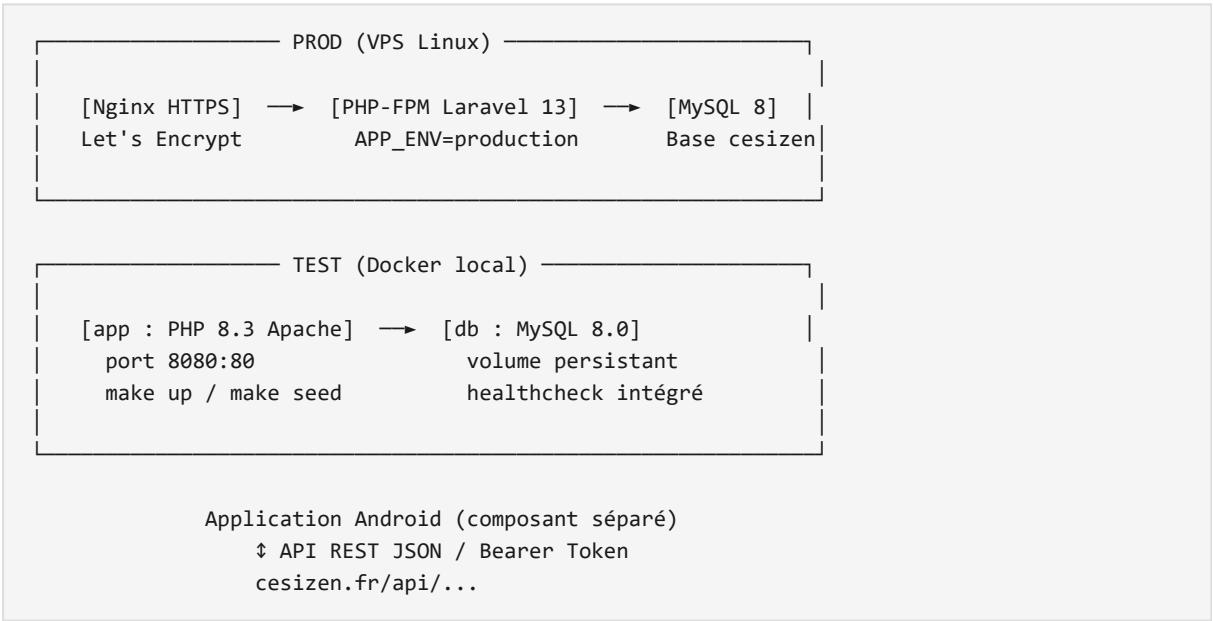
2. Plan de déploiement

2.1 Description des trois environnements et ressources nécessaires

Environnement	Objectif	Infrastructure	Ressources	Accès
DEV	Développement quotidien, expérimentation	WAMP local Windows — PHP 8.3, MySQL 8	PC Windows, 8 Go RAM, WAMP 3.x	Développeur — localhost
TEST 	Validation fonctionnelle, recette, démo	Docker Compose local — PHP 8.3 Apache + MySQL 8	Docker Desktop, 4 Go RAM alloués minimum, 3 Go disque	Équipe — localhost:8080
PROD	Mise à disposition du public	VPS Linux Debian 12 — Nginx + PHP-FPM + MySQL — HTTPS	2 vCPU, 4 Go RAM, 50 Go SSD, IP fixe, nom de domaine	Public — cesizen.fr

L'environnement **TEST est opérationnel via Docker Compose**. La production est décrite architecturalement mais reste théorique dans le cadre de ce projet de formation.

2.2 Architecture technique



2.3 Stratégie de versioning

Branche	Rôle	Protection
main	Code stable, livrable en production	Merge via Pull Request validée uniquement
develop	Intégration continue des développements	Tests CI obligatoires
feature/*	Fonctionnalité isolée	Créée depuis develop, mergée dans develop

Conventions de commit (Conventional Commits)

Préfixe	Usage	Exemple
feat	Nouvelle fonctionnalité	feat(auth): ajouter la réinitialisation du mot de passe
fix	Correction de bug	fix(login): corriger l'erreur 500 après 3 tentatives
ci	Configuration CI/CD	ci: ajouter le build Docker dans le pipeline
docs	Documentation	docs: mettre à jour le README
chore	Maintenance	chore: mettre à jour Laravel 13.x

Tags de version (Semantic Versioning)

- v1.0.0 — Livraison initiale (authentification + information + respiration)
- v1.1.0 — Nouvelle fonctionnalité (ex : historique de sessions)
- v1.0.1 — Correctif de bug ou de sécurité

2.4 Pipeline CI/CD GitHub Actions

Déclenché à chaque push sur `main` ou `develop`, et sur chaque Pull Request vers `main`.

```

Push sur GitHub
  |
  ▼
JOB 1 – Tests Pest (ubuntu-latest)
1. Checkout du code
2. PHP 8.3 + extensions (pdo, pdo_sqlite, mbstring, zip, bcmath)
3. Cache Composer (accélération des runs suivants)
4. composer install --no-progress --prefer-dist
5. cp .env.example .env && php artisan key:generate
6. composer test → 44 tests, SQLite en mémoire, ~2 secondes
   |
   ▼ (seulement si les tests passent – critère de go/no-go)
JOB 2 – Build image Docker (ubuntu-latest)
1. Checkout du code
2. docker build -t cesizen:{SHA_COMMIT} .
   → PHP 8.3 Apache + Node 20 + Vite build + Composer prod

```

Critère de go/no-go : si les 44 tests échouent, le Job 2 ne démarre pas. Aucun code défaillant ne peut atteindre l'environnement de test. C'est la garantie d'intégrité du pipeline.

2.5 Configuration Docker

Commande	Action
<code>make up</code>	Build de l'image + démarrage de l'app et de la base de données
<code>make seed</code>	Migrations + insertion des données de démo
<code>make logs</code>	Affichage des logs en direct
<code>make shell</code>	Accès au terminal dans le conteneur
<code>make down</code>	Arrêt de l'environnement
<code>make reset</code>	Suppression des données et redémarrage propre

2.6 Plan de déploiement étape par étape

Environnement DEV (WAMP local)

1. Cloner : `git clone https://github.com/louisbeaujoin/cesizen.git`
2. Dépendances : `composer install && npm install`
3. Configuration : `cp .env.example .env && php artisan key:generate`
4. Base de données : renseigner `DB_*` dans `.env`, puis `php artisan migrate --seed`
5. Lancement : `composer dev` (serveur PHP + Vite en parallèle)

Environnement TEST (Docker Compose)

1. Prérequis : Docker Desktop installé, 4 Go RAM minimum alloués dans les paramètres Docker
2. Cloner le dépôt
3. Lancer : `make up` (build + démarrage automatique)
4. Données de démo : `make seed`
5. Accès : <http://localhost:8080>

Environnement PROD (VPS Linux — théorique)

1. Provisionner un VPS Debian 12 (2 vCPU, 4 Go RAM, 50 Go SSD)
2. Installer Nginx, PHP 8.3-FPM, MySQL 8, Node.js 20
3. Configurer le virtualhost Nginx : DocumentRoot sur `/public`
4. Certificat SSL : `certbot --nginx -d cesizen.fr`
5. Cloner + installer les dépendances en mode production (`--no-dev`)
6. Configurer `.env` (`APP_ENV=production`, `APP_DEBUG=false`)
7. Migrer : `php artisan migrate --force`
8. Optimiser : `php artisan config:cache && php artisan route:cache`
9. Sauvegardes : cron quotidien `mysqldump` → stockage externe

2.7 Procédure de rollback

En cas d'échec d'un déploiement ou de régression critique détectée après une mise en production, la procédure de rollback est la suivante :

Rollback code — retour à la version précédente

```
# 1. Identifier le tag de la version précédente stable
git log --oneline --tags

# 2. Créer un commit de revert vers le tag précédent
git revert HEAD --no-edit
git push origin main

# Ou retour direct vers un tag précis (ex : v1.0.0)
git checkout -b hotfix/rollback-v1.0.0 v1.0.0
git push origin hotfix/rollback-v1.0.0
# → Ouvrir une PR urgente vers main
```

Rollback base de données — restauration depuis la sauvegarde

```
# Annuler la dernière migration (si applicable)
php artisan migrate:rollback

# Ou restaurer depuis la sauvegarde mysqldump quotidienne
mysql -u cesizen -p cesizen < backup_AAAA-MM-JJ.sql
```

Critères de déclenchement du rollback

Situation	Action	Délai cible
Erreur 500 sur une page critique après déploiement	Rollback code immédiat	< 30 min
Migration échouée en cours de déploiement	<code>migrate:rollback</code> + correction	< 15 min
Corruption de données détectée	Rollback base depuis sauvegarde J-1	< 2h
Performance dégradée de > 50 %	Analyse puis rollback si non corrigeable	< 4h

Prévention : les tags Semantic Versioning permettent d'identifier immédiatement la version stable précédente. Les sauvegardes quotidiennes (mysqldump en cron) garantissent une fenêtre de restauration de données à J-1 maximum.

3. Plan de maintenance

3.1 Outil de ticketing

Le projet utilise **GitHub Issues + GitHub Projects** comme outil unique de ticketing et de suivi. Ce choix centralise le code, la CI/CD et les tickets dans un seul outil, ce qui maximise la traçabilité : chaque ticket est lié au commit et à la Pull Request qui le corrigent.

Élément configuré	Détail
Labels de type	bug · evolution · maintenance · dependabot
Labels de criticité	bloquant-critique · bloquant-fort · majeur · mineur
Labels de statut	en-cours · en-test
Templates d'issue	Déclaration d'incident + Demande d'évolution (pré-remplis)
Board Projects	4 colonnes : À traiter → En cours → En test → Terminé
Template Pull Request	Checklist : tests, sécurité, revue de code
Dependabot	PRs automatiques hebdomadaires (Composer + npm + Actions)

3.2 Méthodologie de gestion des tickets

Cycle de vie d'un ticket



SLA — Niveaux de service contractuels

Niveau	Critère	Prise en compte	Correction	Exemple
Bloquant critique	Application inaccessible / données compromises	1h ouvrée	3h ouvrées	Serveur hors ligne, fuite de données
Bloquant fort	Fonctionnalité majeure KO, sans contournement	2h ouvrées	6h ouvrées	Connexion impossible
Majeur	Gêne significative avec contournement possible	7h ouvrées	16h ouvrées	Affichage incorrect sur mobile
Mineur	Inconfort visuel ou ergonomique	1 jour ouvré	40h ouvrées	Faute d'orthographe, coquille CSS

Communication prestataire / Ministère

- **Canal principal** : GitHub Issues — traçabilité complète et historique permanent
- **Canal d'urgence** : email direct pour les incidents bloquants critiques
- **Compte-rendu mensuel** : export du board GitHub Projects (tickets ouverts / fermés)
- **Réunion mensuelle** : revue des tickets, priorisation des évolutions à venir

3.3 Veille technologique

Source	Fréquence	Objet
Dependabot (GitHub)	Automatique — hebdomadaire	MAJ Composer + npm + GitHub Actions
Laravel News (laravel-news.com)	Hebdomadaire	Nouvelles fonctionnalités, breaking changes
CHANGELOG Laravel (GitHub)	À chaque release	Détail des corrections et nouveautés
Bulletins ANSSI / CVE	Hebdomadaire	Alertes de sécurité critiques
PHP RFC	Mensuelle	Évolutions du langage PHP

Processus : Dependabot ouvre une PR automatique → le prestataire lit le CHANGELOG → les tests CI s'exécutent → merge si tout est vert → tag de version si nécessaire.

4. Plan de sécurisation

4.1 Protections natives de Laravel

L'analyse du code réel (contrôleurs, routes, middleware, vues Blade) confirme que Laravel couvre automatiquement les vulnérabilités suivantes :

- **Injection SQL (OWASP A03)** : Eloquent ORM utilise des requêtes préparées — aucune requête SQL brute dans le code.
- **XSS HTML (OWASP A03)** : Blade `{{ }}` applique `htmlspecialchars()` automatiquement sur toutes les variables.
- **CSRF (OWASP A01)** : Token `@csrf` présent dans chaque formulaire POST/PUT/DELETE, vérifié par le middleware.
- **Hashage mots de passe (OWASP A02)** : Cast `'hashed'` sur le modèle User + `Hash::make()` explicite. Algorithme : `bcrypt` 12 rounds.
- **Contrôle d'accès admin (OWASP A01)** : Double middleware `auth` + `admin` sur toutes les routes admin. Protection anti-auto-suppression.
- **Fixation de session (OWASP A07)** : `session()->regenerate()` après chaque connexion réussie.

4.2 Matrice de risques

#	Risque	Probabilité /5	Impact /5	Criticité /25	Statut
R1	Brute force sur le login	4	5	20	✓ Corrigé
R2	XSS via paramètres URL → <script>	3	4	12	✓ Corrigé
R11	Panne serveur / perte de données	2	5	10	⚠ Plan sauvegarde prod + rollback
R4	APP_DEBUG=true en production	3	3	9	✓ Corrigé
R3	Token API stocké en clair en base	2	4	8	⚠ Hachage recommandé v1.1
R5	Absence d'en-têtes HTTP de sécurité	4	2	8	✓ Corrigé
R6	Injection SQL	1	5	5	✓ Eloquent natif
R9	Mots de passe faibles en base	1	5	5	✓ bcrypt natif
R10	Escalade de privilèges admin	1	5	5	✓ Middleware natif
R12	Fuite de données personnelles	1	5	5	✓ RGPD + sécurité
R7	XSS HTML	1	4	4	✓ Blade natif
R8	CSRF	1	4	4	✓ @csrf natif

10 risques sur 12 couverts = 83 % — objectif 75 % atteint et dépassé

4.3 Correctifs appliqués dans le code

Risque	Fichier modifié	Correction
Brute force login (R1)	routes/web.php · routes/api.php	Middleware throttle:5,1 sur POST /connexion et POST /api/login — 5 tentatives / minute / IP
XSS paramètres URL (R2)	app/Http/Controllers/BreathingController.php	Cast (int) sur inspiration, apnée, expiration et cycles avant injection dans <script>
APP_DEBUG en prod (R4)	.env.example	Valeur changée de true à false dans le template d'environnement
En-têtes HTTP absents (R5)	app/Http/Middleware/SecurityHeadersMiddleware.php	Nouveau middleware ajouté sur toutes les routes web : X-Content- Type-Options, X- Frame-Options, Referrer-Policy, Permissions-Policy

4.4 Volet RGPD

Données personnelles collectées

Catégorie	Données	Table	Finalité	Conservation
Identité	Nom, e-mail	users	Authentification	Durée du compte + 1 an
Sécurité	Mot de passe (haché bcrypt)	users	Contrôle d'accès	Durée du compte
Santé mentale	Sessions de respiration	breathing_sessions	Suivi personnel	2 ans glissants
Technique	Identifiant de session	sessions	Maintien connexion	120 minutes

Point d'attention : les données d'historique de respiration peuvent être qualifiées de données de santé (Article 9 RGPD). En production réelle, un hébergement certifié HDS (Hébergeur de Données de Santé) serait requis. Dans le cadre de ce projet, les données sont pseudonymisées.

Droits des utilisateurs

Droit (RGPD)	Implémentation dans CESIZen
Accès (Art. 15)	Page profil /profil — consultation nom et e-mail
Rectification (Art. 16)	Modification du profil (PUT /profil)
Effacement (Art. 17)	Suppression par l'administrateur
Limitation (Art. 18)	Désactivation de compte par l'admin (is_active=false)
Portabilité (Art. 20)	À implémenter en v1.1.0 : export CSV des sessions de respiration

4.5 Plan de gestion de crise

Étape	Délai	Action	Responsable
1. Détection	T + 0	Monitoring, alerte Dependabot ou signalement utilisateur	Prestataire
2. Qualification	T + 1h max	Évaluation gravité, estimation du périmètre (données exposées ?)	Prestataire
3. Notification	T + 2h max	Email urgent au DSI et au DPO du Ministère avec rapport d'incident	Prestataire
4. Isolation	T + 2h	Mise hors ligne du serveur compromis si nécessaire / rollback si régression	Prestataire + DSI
5. CNIL	T + 72h max	Déclaration obligatoire sur notifications.cnil.fr si données personnelles exposées	DPO Ministère
6. Communication	T + 72h max	Email aux utilisateurs concernés si risque élevé pour leurs droits	Ministère
7. Post-incident	T + 7 jours	Analyse des causes, plan de remédiation, mise à jour matrice de risques	Prestataire

4.6 Bonnes pratiques de développement

Pratique	Mise en œuvre	Preuve vérifiable
Architecture MVC	Séparation Controllers / Models / Views	Structure app/ + resources/views/
Documentation du projet	README.md personnalisé : installation, commandes, architecture	Racine du dépôt GitHub
.env hors du dépôt	.env dans .gitignore ligne 3	<code>git log -- .env</code> → aucun résultat
Conventions de code	Laravel Pint (PSR-12)	laravel/pint dans composer.json
Tests automatisés	44 tests Pest, exécutés en CI	<code>composer test</code> → 44 passed
Revue de code	Template PR avec checklist	.github/PULL_REQUEST_TEMPLATE.md
Dépendances surveillées	Dependabot hebdomadaire	.github/dependabot.yml
Versioning sémantique	Tags v1.0.0 sur les releases	<code>git tag</code> → v1.0.0

5. Conclusion

Ce dossier présente l'ensemble du plan de déploiement, de maintenance et de sécurisation de la plateforme CESIZen. Les choix techniques privilégient la simplicité, la traçabilité et l'explicabilité, conformément aux exigences du projet Ministère (budget 75 000 €, livraison 12 mois).

L'environnement de test est opérationnel via Docker Compose, le pipeline CI/CD exécute automatiquement les 44 tests à chaque push, et **83 % des risques identifiés ont fait l'objet d'une action corrective ou préventive**, au-delà du seuil de 75 % requis.

Points forts de la solution

- **Un seul outil** pour code + CI/CD + tickets (GitHub) — traçabilité maximale, friction minimale
- **Tests automatisés bloquants** — critère de go/no-go : aucun code cassé ne peut atteindre l'environnement de test
- **Procédure de rollback définie** — retour à l'état stable en moins de 30 minutes en cas de régression
- **Sécurité en couches** — Laravel nativement + correctifs ajoutés (throttle, cast, en-têtes HTTP)
- **RGPD intégré dès la conception** — minimisation des données, droits utilisateurs, durées de conservation

Perspectives d'évolution

- Déploiement continu automatisé vers la production (CD) avec approbation manuelle
- Monitoring applicatif (Sentry pour les erreurs, Uptime Robot pour la disponibilité)
- Export des données personnelles (droit à la portabilité RGPD, Article 20) — v1.1.0
- Hachage des tokens API (R3) — v1.1.0
- Hébergement certifié HDS pour la mise en production réelle